



Introductory MATLAB

MTE 204 Tutorial

Devon C. Hartlen

2021-09-22

Previous Programming Experience

- What is expected:
 - Understand the concept of loops (`for` and `while`)
 - Understand the concept of conditionals (`if`)
 - Understand the concept of Boolean logic (i.e. `a == b`, `a > b`, etc.)
 - Understand the concept of assigning variables
 - Understand the concept of floats vs integers
- What is not expected:
 - Previous MATLAB knowledge

MATLAB – An Introduction

- Designed specifically for numerical computing
 - Specifically matrices and linear algebra
 - MATrix LABoratory
 - Also contains built-in functions for optimization, root finding, differential equations, etc.
- Big differences from other programming languages
 - Arrays index from 1, not 0
 - Makes sense for mathematics
 - Completely opposite from many other programming languages
 - No need to define variable type
 - Only runs within MATLAB environment (no independent compilers)

MATLAB – Other Resources

- MathWorks ‘Onramp’ course. A 2-3 hour, interactive, self-directed course that is run entirely within a browser
 - <https://www.mathworks.com/learn/tutorials/matlab-onramp.html>
- For help with specific MATLAB functions
 - `doc function` in command window
- To look up a keyword in MATLAB (in case you don’t know a specific command)
 - `lookfor keyword` in command window
- Other important resource:
 - The course TAs! It is what we are paid for after all. Access through Piazza or email.
 - Google! Most common results are Stack Exchange and MathWorks forums.

Accessing MATLAB

- Create a Mathworks account first
 - <https://www.mathworks.com/academia/tah-portal/university-of-waterloo-31483447.html>
 - Follow the link
- Two usage options
 - Download MATLAB (5-20 Gb install)
 - MATLAB Online (operates in a browser, recommended for MTE 204)
 - <http://matlab.mathworks.com/>

MATLAB User Interface Overview

The image shows the MATLAB R2020b - academic use interface. The top menu bar includes HOME, PLOTS, APPS, EDITOR, PUBLISH, and VIEW. The toolbar contains icons for file operations (New, Open, Save, Find Files, Compare, Print), navigation (Go To, Find), editing (Insert, Comment, Indent), breakpoints, and running code (Run, Run and Advance, Run Section, Advance, Run and Time). The main workspace is divided into several windows:

- File Explorer** (blue border): Located on the left, showing the current folder structure. It includes a list of files and folders, such as Functions, TestCases, ThirdPartyFunctions, .gitignore, arcgen.m, PreProcessInputSignals.m, and README.md.
- Script Editing Window** (orange border): The central window for writing and executing scripts and functions. It displays a script named T01Q02.m with the following content:

```
1 % MTE 204 - Numerical Methods
2 % Tutorial 01, Question 02
3 % Code by: Devon C. Harten
4 %
5 % Determine the largest root of the following equation a) graphically, and
6 % b) using Newton-Raphson Method.
7 % f(x) = x^3 - 6x^2 + 11x - 6.1
8
9 % Initialization
10 close all; % closes all plot windows
11 clear; % clears variables from memory
12 clc; % clear the command window
13
14 %% Part a - Graphical Method
15 % Initialize x array to be used for plotting. linspace(a,b,n) creates an
16 % array of n points between a and b
17 x = linspace(0,4,100);
18 % Compute the function at all points in x. In this problem, we forgo using
19 % a for loop to make use of Matlab's element-wise operators. Using .* and
20 % .^ performs multiplication and exponents for every element in the initial
21 % array. Addition and subtraction do not need a dot.
22 y = x.^3 - 6.*x.^2 + 11.*x - 6.1;
23 % Create the plot
24 figure(); % This forces matlab to create a new figure
25 hold on; % by default matlab overwrites every plot its makes. This stops that
26 % create a line plot
27 plot(x,y)
28 grid on; % enables grid lines on the plot
29 % Set x and y plot limits. Matlab sets these automatically otherwise
30 xlim([0,4])
31 ylim([-6,6])
32
33 %% Part b - Newton-Raphson Method
34 xOld = 3.5; % Set initial guess
35 iterMax = 3;
36 % Begin iterations. Use for loop because we are limiting to 3 iterations
37 for iter = 1:iterMax+1
38     % compute function and derivative at x_i
39     f = xOld^3 - 6*xOld^2 + 11*xOld - 6.1;
40     fPrime = 3*xOld^2 - 12*xOld + 11;
41
42     % Compute x_i+1
43     xNew = xOld - f/fPrime;
```
- Command Window** (green border): Located on the right, used for minor command entry and viewing outputs. It shows the prompt `>> |`.
- Workspace** (yellow border): Located at the bottom left, used for viewing variables. It has columns for Name and Value.

Annotations and text overlays include:

- "Your interface may look different!" and "Windows can be dragged to different locations" in the top right.
- "Script Editing Window (write and execute scripts and functions)" in orange text over the script editor.
- "Command Window (minor command entry and viewing outputs)" in green text over the command window.
- "Workspace (view variables)" in yellow text over the workspace window.
- "File Explorer" in blue text over the file explorer window.

Defining Variables

- Scalar values:
 - `a = 6`
 - `b = 5.5`
 - `c = 1.253e-3`
- Text (characters)
 - `d = 'this is a character array'`
- Semicolon special character, `;`
 - If a command is terminated with a semicolon, the result will not be printed to the command window.

Arrays – Creation

- Arrays are the exact same as matrices
- In MATLAB, rows vs columns matter
- To create matrices, use square brackets, separate entries in a row with commas, separate rows with semicolons
 - `a = [ 1, 2, 3; 4, 5 ,6; 7, 8, 9]`
- Alternatively, spaces can separate entries in a row, new lines can be used after semicolons
 - `b = [9 8 7;
6 5 4;
3 2 1]`
- Special matrix creation commands
 - `c = eye(3)` Creates an identity matrix (3x3 in this case)
 - `d = ones(n,m)` Creates an n x m array of ones
 - `e = zeros(n,m)` Creates an n x m array of zeros
 - `f = rand(n,m)` creates an n x m array of random floats between (0,1)

Array – Creation and Indexing

- A common task is creating an array with equally spaced values between two points. Very important for plotting!
- Two common methods
 - `a = 0:0.1:1` Creates an array with values between 0 and 1 spaced at 0.1 apart
 - `b = linspace(0,1,100)` Creates an array with 100 values between 0 and 1
- Indexing is the act of getting a value from an array. MATLAB indexes with regular parenthesis.
 - `Array(row, column)` This command extracts the entry located in row 4
 - `c(3, 4)` Extracts the entry from the 3rd row and 4th column
 - `d(3, :)` Extract all entries in the 3rd row
 - `e(:, 4)` Extracts all entries in the 4th column

M-files (Scripts and Functions)

- M-files are used to create MATLAB programs
 - To be used for all projects and assignments.
- M-files come in two forms: scripts and functions
- Scripts can be thought of as traditional programs. They are just an assemblage of commands
 - When scripts execute, all variables are saved in memory after execution
- Functions are m-files that can be called by other scripts/functions
 - Must start with the following line
 - `function [output1, output2]=functionName(input1, input2,...)`
 - Functions take arguments, and return only specific outputs. Intermediate values are not saved to memory after execution.

A note on MATLAB memory

- By default, MATLAB does not delete variables before running a new script
- By default, defined variables are overwritten
- This can cause problems when the same program is run with different values
- MATLAB memory can be cleared with the `clear` (no arguments)
- I recommend starting all scripts with `clear` so variables from previous scripts, commands, etc. do not interfere with the current program.
- The command window can be cleared with the `clc` command

if Statements and Boolean Algebra

- if statements take a boolean argument
 - Examples of Boolean operators: >, <, >=, <=
 - MATLAB oddities: == (equals), ~= (not equal)
- Example if statement with multiple conditions

```
if (x >= 0) && (x < 5)
    do something;
elseif (x >= 5) && (x <= 10)
    do something else;
else
    do something if x does not fit any if or elseif statement;
end
```

Loops

- MATLAB has two options for loops: `for` and `while`
 - All loop
- `for` loops proceed for a set number of iterations. Loop commands are terminated with an `end` command. Indentation is recommended for readability, but is not required.

```
for I = 1:10
    commands to perform
end
```

Loops – Cont.

- `while` loops will loop until a specific condition is no longer met. `while` loops are terminated with the `end` command.

```
while true  
    commands to repeat indefinitely;  
end
```

```
while iteration <= maxIterations  
    command to repeat;  
    iteration = iteration + 1;  
end
```

- multiple conditions can be set using `&&` (and) or `||` (or)

Loops – General Notes

- A specific iteration can be skipped using the `continue` command. Useful if you want to skip calculations at a specific iteration

```
for i=1:10
    if i==2
        continue; % skips the 2nd iteration
    end
end
```

- To exit a loop, use the `break` command

```
for i=1:10
    if i==2
        break; % exits the loop on the 2nd iteration
    end
end
```

- To force MATLAB to stop a program, hit `ctrl+c` on the keyboard. Useful for infinite loops.

Example Problem – For Loops

- Create a script to compute the Fibonacci sequence to an arbitrary number of terms.
- Recall the Fibonacci sequence is defined as

$$F_i = F_{i-1} + F_{i-2}$$
$$F_0 = 0, \quad F_1 = 1$$

- See script “ForLoopExample.m”

Example Problem – While Loop

- Divide an arbitrary initial value x_0 by a factor repeatedly until the value falls below a given tolerance or a maximum number of iterations is exceeded.
- See script “WhileLoopExample.m”

Plotting

- A key feature of MATLAB is highly integrated plotting features. MATLAB has a large array of plots, but lets focus on the simplest
- `plot(x, y)` creates line plots of two 1-D arrays
- Aspects of the line plot can be edited with arguments after the two arrays. Multiple arguments can be used.
 - `plot(x, y, '-o')` places dots at every (x,y) point
 - `plot(x, y, 'LineWidth', 2.0)` Creates a line with width 2.0
 - Enter `doc plot` into the command window to see the full MATLAB documentation for plotting. There are a huge number of line types, color options, etc.

Plotting – Cont.

- By default, MATLAB overwrites open plots. Therefore, two `plot` commands will only produce a plot with a single line plot
- To override this behaviour, place `hold on` before the first `plot` command
- By default, MATLAB reuses a single open plot windows. But what if we want separate plot windows?
- Place `figure()` whenever you want the following plotting commands to appear in a new window. Must specify `hold on` for each `figure()` call
- To close all open plot windows, run the command `close all`

Plotting – Specify Axis Labels and Limits

- By default, MATLAB guesses at plot limits and does not include labels
- Plot limits
 - `xlim([lower, upper]), ylim([lower, upper])`
 - note the use of parenthesis and square brackets. The parenthesis are used to define the arguments of `xlim`, the square brackets define a 2 element array which actually specify plot limits. Confusing, but this is just how MATLAB works...
 - If only a single limit command is entered, the other axes is computed based on specified limits
- Plot Labels
 - `xlabel('label for x-axis'), ylabel('label for y-axis')`
 - `title('This title is placed overtop the plot')`
 - `legend('entry for plot 1', 'entry for plot2', 'etc.')`
- Add a grid to your plots: `grid on`

General Example

- Create a plot of with the two following functions on the interval $x = (0,2)$.
 - $x^3 - 10x^2 + 4x + 2$
 - $2x + 2 \sin(10x)$
- What do we need to do to accomplish this?
 - Define an array x with a lot of points between (0,2)
 - Compute function at all elements of the array x
 - Create a plot
- See script “PlottingExample.m”

Element-Wise Operations

- In the previous example, we used a `for` loop to compute the function at every point in the `x` array
- MATLAB has class of operators that allow us to perform the same operation on all elements of an array simultaneously.
 - These operators are `+`, `-`, `.*`, `./`, and `.^`

- The loop:

```
for i = 1:length(x)
    y(i) = x(i)^3 - 10*x(i)^2 + 4*x(i) - 2;
end
```

- Can be replaced with this single line of code:

- `y = x.^3 - 10.*x.^2 + 4.*x - 2;`